LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 14. Functions

Lecture: 159. Local & Global Variables

Section 1: Lecture Summary

This lecture explains the concepts of **local and global variables** in Python functions using example-driven demonstrations. It starts by defining variables inside and outside functions and explores where they are accessible and modifiable. The instructor shows that variables declared inside a function are local and available only within that function, while variables declared outside any function are global and readable within functions but cannot be modified directly unless explicitly declared as `global`. The lecture covers the significance of defining global variables **before** function calls to avoid errors. Lastly, it introduces Python's built-in functions `locals()` and `globals()` to introspect current local and global symbol tables, respectively, revealing variable information in dictionary form. Emphasis is on understanding scope, accessibility, and modification rules for variables inside and outside functions.

Section 2: Key Concepts and Explanations

- **Local Variables**

  - Declared **inside** a function.

  - Accessible **only inside** that function.

  - Example: `a = 10` inside a function means `a` is local.

  - Local variables cannot be accessed outside their defining function.

- **Global Variables**

  - Declared **outside** of all functions, at the script/module level.

  - Accessible **anywhere** in the program, including inside functions.

  - Can be **read** inside functions freely.

  - Cannot be **modified** inside functions unless declared with the `global` keyword.

  - Must be declared **before a function call** to be accessible inside that function; else a `NameError` occurs.

- **Modifying Global Variables inside Functions**

  - Assigning a value to a global variable name inside a function **creates a local variable** unless `global varname` is declared first.

  - To modify a global variable inside a function, declare `global varname` at the beginning of the function.

- **Order of Execution**

  - Code outside functions runs in order.

  - Functions run only when called.

  - Variables must be declared before the function call to be accessible.

- **Built-in functions for variable inspection**

  - `locals()` returns a dictionary of all local variables visible in the current scope.

  - `globals()` returns a dictionary of all global variables visible to the current module, including predefined Python globals.

Section 3: Example Code and Use Cases

Example 1: Local variable inside a function

Output: local: 10

```
def fun():
    a = 10          # local variable
    print('local:', a)

fun()
```

Explanation: `a` is local to `fun()`. It is printed when `fun` runs.

Example 2: Global variable accessed inside function

```
g = 5.25            # global variable declared before function

def fun():
    a = 10          # local variable
    print('local:', a)
    print('global:', g)  # reading global variable inside function

print('Outside-1:', g)
```

```
fun()
print('Outside-2:', g)
```

Output:

```
Outside-1: 5.25
local: 10
global: 5.25
Outside-2: 5.25
```

Explanation: `g` is global and readable inside and outside the function.

Example 3: Attempting to modify global variable inside function without global keyword

```
g = 5.25

def fun():
    a = 10
    g = 199       # creates new local g, does not change global g
    print('local:', a)
    print('global (local g):', g)

print('Outside-1:', g)
fun()
print('Outside-2:', g)
```

Output:

```
Outside-1: 5.25
local: 10
global (local g): 199
Outside-2: 5.25
```

Explanation: `g = 199` inside the function creates a new local variable `g`, global `g` remains unchanged.

Example 4: Modifying global variable inside function using global keyword

```
g = 5.25

def fun():
    global g      # declare g as global
    a = 10
    g = 199       # modifies global g
    print('local:', a)
    print('global:', g)

print('Outside-1:', g)
```

```
fun()
print('Outside-2:', g)
```

Output:

```
Outside-1: 5.25
local: 10
global: 199
Outside-2: 199
```

Explanation: Using `global g` lets the function modify the global variable `g`.

Example 5: Variable declared after function call causes error

```
def fun():
    print(g)     # attempt to print global variable g

fun()
g = 5.25         # declared after fun() is called
```

Output:

```
NameError: name 'g' is not defined
```

Explanation: `g` must be declared before function call to be accessible inside `fun()`.

Example 6: Using locals() and globals()

```

x, y, z = 5, 1.25, "hi"   # global variables

def fun():
    a, b, c = 1, 2, 3     # local variables
    print("Locals:", locals())    # dictionary of local variables
    print("Globals:", globals())  # dictionary of global variables

fun()
```

Sample Output (locals() and globals() dictionary format):

```
Locals: {'a': 1, 'b': 2, 'c': 3}
Globals: {..., 'x': 5, 'y': 1.25, 'z': 'hi', 'fun': <function fun at ...>}
```

Explanation: `locals()` returns a dict of local variables; `globals()` returns all global variables including Python predefined globals.

Section 4: Key Takeaways

- Variables inside functions are **local**; variables outside are **global**.

- Local variables are accessible **only inside** their function.

- Global variables are accessible **anywhere**, including inside functions.

- Functions **can read global variables** but **cannot modify** them unless explicitly declared with `global`.

- Declaring a variable `global` inside a function lets you **modify the global variable**.

- Global variables must be declared **before** the function call to be visible inside the function.

- Calling a function runs only the code inside it; code outside runs sequentially.

- Use `locals()` to see current local variables as a dictionary.

- Use `globals()` to see global variables (and some built-in globals) as a dictionary.

- Understanding scope and variable lifetimes is crucial for proper Python function programming.